



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика, искусственный интеллект и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе № 1

по курсу «Методы вычислений»

на тему: «Венгерский метод решения задачи о назначениях»

Вариант № 4

Студент ИУ7-13М
(Группа)

(Подпись, дата)

Орду М. А.
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Власов П. А.
(И. О. Фамилия)

2025 г.

1 Теоретическая часть

1.1 Содержание работы

Цель работы: изучение венгерского метода решения задачи о назначениях.

1. реализовать венгерский метод решения задачи о назначениях в виде программы на ЭВМ;
2. провести решение задачи с матрицей стоимостей, заданной в индивидуальном варианте, рассмотрев два случая:
 - (а) задача о назначениях является задачей минимизации,
 - (б) задача о назначениях является задачей максимизации.

1.2 Постановка задачи

В распоряжении работодателя имеется n работ и n исполнителей. Стоимость выполнения i -ой работы j -ым исполнителем составляет $c_{ij} \geq 0$ единиц. Требуется распределить работы между исполнителями так, чтобы:

1. каждый исполнитель выполнял ровно одну работу;
2. общая стоимость выполнения всех работ была минимальной.

1.2.1 Математическая формулировка

Обозначим $C = (c_{ij})_{i,j=\overline{1;n}}$, где C – матрица стоимостей. Введем управляемые переменные:

$$x_{ij} = \begin{cases} 1, & \text{если } i\text{-ую работу выполняет } j\text{-ый работник} \\ 0, & \text{иначе} \end{cases}, \quad (1.1)$$

где $i, j = \overline{1, n}$

Обозначим $X = (x_{ij}), i, j = \overline{1, n}$, где X – матрица назначений.

Тогда общая стоимость выполнения работ:

$$f = \sum_{i=1}^n \sum_{j=1}^n c_{ij} \cdot x_{ij} \quad (1.2)$$

Условие того, что j -й работник выполняет ровно одну работу:

$$\sum_{i=1}^n x_{ij} = 1, \quad (1.3)$$

где $j = \overline{1; n}$

Условие того, что i -ю работу выполняет один работник:

$$\sum_{j=1}^n x_{ij} = 1, \quad (1.4)$$

где $i = \overline{1; n}$

Тогда математическая постановка задачи о назначениях имеет следующий вид:

$$\left\{ \begin{array}{l} f = \sum_{i=1}^n \sum_{j=1}^n c_{ij} \cdot x_{ij} \rightarrow \min \\ \sum_{i=1}^n x_{ij} = 1, \quad j = \overline{1; n} \\ \sum_{j=1}^n x_{ij} = 1, \quad i = \overline{1; n} \\ x_{i,j} \in 0, 1, \quad i, j = \overline{1; n} \end{array} \right. , \quad (1.5)$$

где f – целевая функция, c_{ij} – элементы матрицы стоимостей C , x_{ij} – элементы матрицы назначений X .

1.2.2 Исходные данные индивидуального варианта

$$C = \begin{bmatrix} 3 & 5 & 2 & 4 & 8 \\ 10 & 10 & 4 & 3 & 6 \\ 5 & 6 & 9 & 8 & 3 \\ 6 & 2 & 5 & 8 & 4 \\ 5 & 4 & 8 & 9 & 3 \end{bmatrix}$$

1.3 Описание венгерского метода

Венгерский метод решения задачи о назначениях применяется для решения задачи минимизации, то есть для случая, когда матрица стоимостей C содержит в себе стоимости работ. Также матрица стоимостей C может использоваться для решения задачи максимизации, в этом случае ее элементы отражают прибыль работодателя от назначения j -го работника на i -ю работу.

Решение задачи максимизации отличается от решения задачи минимизации тем, что в данном случае в начале алгоритма добавляется шаг, на котором необходимо найти максимальный элемент столбца, вычесть его из каждого элемента матрицы и домножить матрицу на -1 .

Алгоритм решения задачи о назначениях венгерским методом представлен на рисунке 1.1.

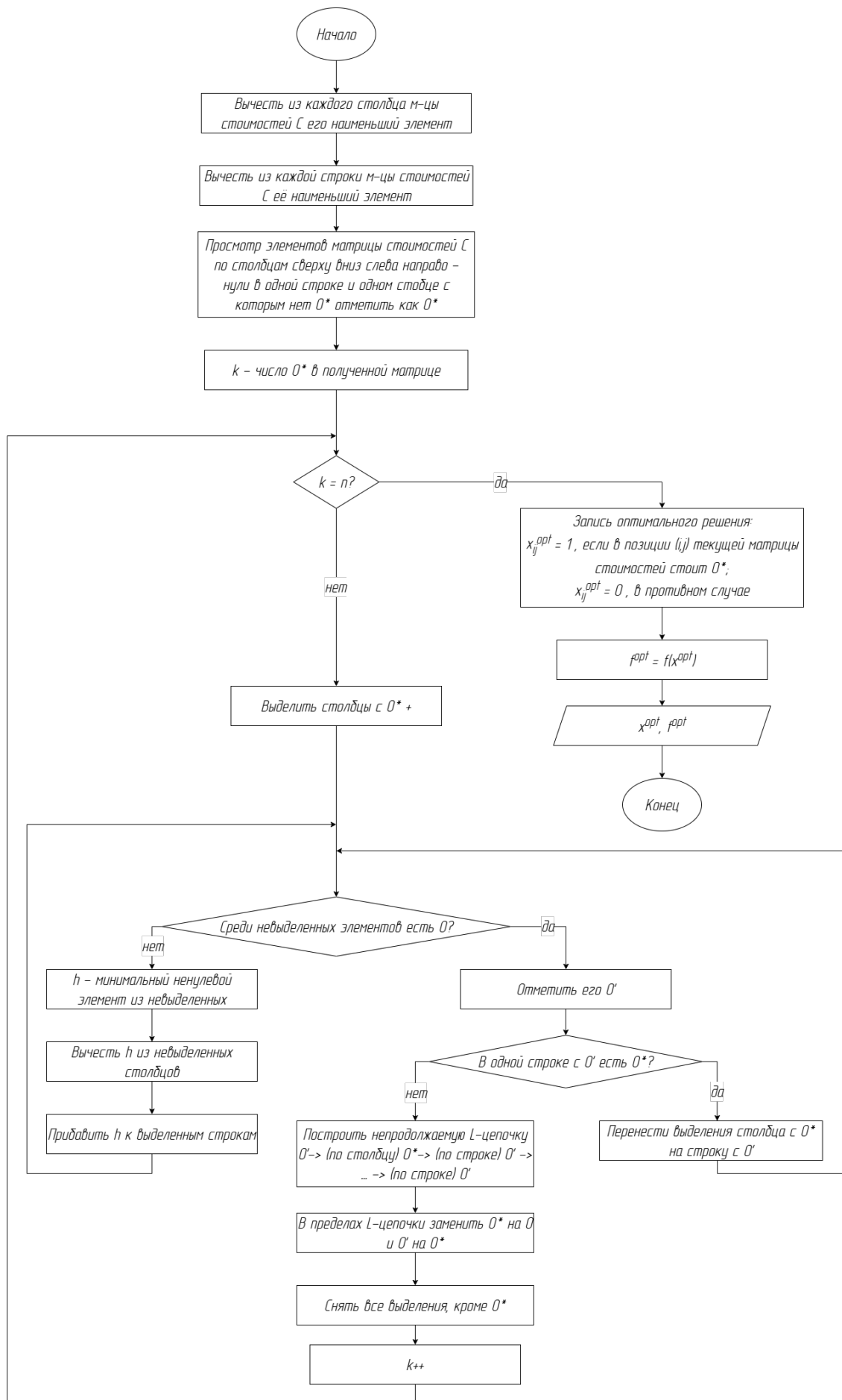


Рисунок 1.1 – Алгоритм решения задачи о назначениях венгерским методом

2 Практическая часть

2.1 Программная реализация

В листингах 2.1 и 2.2 приведен текст программы.

Листинг 2.1 – main.m

```
1 % Лабораторная работа 1
2 % Решение задачи о назначениях венгерским методом
3
4 clear; clc;
5
6 % Параметры
7 debug = true;
8 maximization = false;
9
10 % Исходная матрица стоимостей
11 C = [3, 5, 2, 4, 8;
12      10, 10, 4, 3, 6;
13      5, 6, 9, 8, 3;
14      6, 2, 5, 8, 4;
15      5, 4, 8, 9, 3];
16
17 [X_opt, f_opt] = hungarian(C, debug, maximization);
18
19 disp("Оптимальная матрица назначений X_opt = ");
20 disp(X_opt);
21
22 fprintf('f_opt: %g\n', f_opt);
```

Листинг 2.2 – hungarian.m

```
1 function [X_opt, f_opt] = hungarian(C, debug, maximization)
2
3     [nRows, nCols] = size(C);
4     A = C;
5
6     % --- Приведение задачи к минимизации ---
7     if maximization
8         maxVal = max(C, [], "all");
9         A = -(C - maxVal);
10        if debug
11            disp("Преобразование матрицы для задачи на
```

```

        максимум");
12         disp(A);
13     end
14 end
15
16 % --- Вычитание минимальных элементов по столбцам ---
17 minCol = min(A, [], 1);
18 A = A - minCol;
19 if debug
20     disp("Вычитание минимальных элементов по столбцам:");
21     disp(A);
22 end
23
24 % --- Вычитание минимальных элементов по строкам ---
25 minRow = min(A, [], 2);
26 A = A - minRow;
27 if debug
28     disp("Вычитание минимальных элементов по строкам:");
29     disp(A);
30 end
31
32 % Начальная СНН
33 stars = initStars(A);
34 N = size(A,1);
35 quotes = zeros(N);
36
37 if debug
38     disp("Начальные 0*");
39     showDebug(A, stars, quotes, [], zeros(N));
40 end
41
42 k = nnz(stars);
43 if debug
44     disp("k = ");
45     disp(k);
46 end
47
48 % --- Основной цикл улучшений ---
49 if k < nCols
50     iter = 1;
51     while k < nRows

```

```

52         if debug
53             fprintf("Итерация %d\n", iter);
54         end
55
56         markedCols = findMarkedCols(stars);
57         markedRows = zeros(size(A,2));
58
59         while true
60             [found, col, row] = findFreeZero(A,
61                 markedCols, markedRows);
62
63             if found
64                 quotes(row,col) = 1;
65
66                 if debug
67                     disp("После нахождения 0'");
68                     showDebug(A, stars, quotes,
69                         markedCols, markedRows);
69                 end
70
71                 [hasStar, col2] = hasStarInRow(row, stars);
72
73                 if hasStar
74                     markedRows(row) = 1;
75                     markedCols(col2) = 0;
76
77                     if debug
78                         disp("Изменили выделенные строки и
79                             столбцы");
80                         showDebug(A, stars, quotes,
81                             markedCols, markedRows);
82                     end
83                 else
84                     stars = buildLchain(stars, quotes,
85                         row, col);
86                     k = k + 1;
87                     markedCols = findMarkedCols(stars);
88                     markedRows = zeros(size(A,2));
89
90                     if debug
91                         disp("Новые 0* после построения

```

```

88         L-цепочки:");
89         disp(stars);
90     end
91     break;
92 end
93 else
94     A = recalMatrix(A, markedRows,
95         markedCols);
96     if debug
97         disp("Пересчитали матрицу h");
98         showDebug(A, stars, quotes,
99             markedCols, markedRows);
100     end
101 end
102
103     iter = iter + 1;
104 end
105 end
106
107 % --- Построение оптимального решения ---
108 X_opt = zeros(nCols);
109 for i = 1:nRows
110     for j = 1:nCols
111         if stars(i,j)
112             X_opt(i,j) = 1;
113         end
114     end
115 end
116
117 f_opt = sum(C(logical(stars)));
118 end
119
120 % ----- ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ
121 -----
122 function stars = initStars(A)
123     N = size(A,1);
124     stars = zeros(N);

```

```

125     for j = 1:N
126         for i = 1:N
127             if A(i,j) == 0 && max(stars(:,j))==0 &&
                max(stars(i,:))==0
128                 stars(i,j) = 1;
129                 break;
130             end
131         end
132     end
133 end
134
135 function cols = findMarkedCols(stars)
136     cols = sum(stars);
137 end
138
139 function [found, col, row] = findFreeZero(A, markedCols,
    markedRows)
140     N = size(A,1);
141     found = false;
142     col = 0; row = 0;
143
144     for j = 1:N
145         for i = 1:N
146             if A(i,j) == 0 && ~markedCols(j) && ~markedRows(i)
147                 found = true;
148                 col = j; row = i;
149                 return;
150             end
151         end
152     end
153 end
154
155 function [hasStar, col] = hasStarInRow(row, stars)
156     hasStar = false; col = 0;
157     for j = 1:size(stars,2)
158         if stars(row,j) == 1
159             hasStar = true;
160             col = j;
161             return;
162         end
163     end

```

```

164 end
165
166 function [hasStar, row] = hasStarInCol(col, stars)
167     [hasStar, row] = hasStarInRow(col, stars');
168 end
169
170 function A = recalcMatrix(A, markedRows, markedCols)
171     N = size(A,1);
172     h = inf;
173
174     for j = 1:N
175         for i = 1:N
176             if ~markedCols(j) && ~markedRows(i)
177                 h = min(h, A(i,j));
178             end
179         end
180     end
181
182     for j = 1:N
183         for i = 1:N
184             if ~markedCols(j) && ~markedRows(i)
185                 A(i,j) = A(i,j) - h;
186             elseif markedCols(j) && markedRows(i)
187                 A(i,j) = A(i,j) + h;
188             end
189         end
190     end
191 end
192
193 function newStars = buildLchain(stars, quotes, row, col)
194     newStars = stars;
195     curRow = row; curCol = col;
196     flag = true;
197
198     while flag
199         newStars(curRow, curCol) = 1;
200
201         [flag, curRow] = hasStarInCol(curCol, stars);
202
203         if flag
204             newStars(curRow, curCol) = 0;

```

```

205         [flag, curCol] = hasStarInRow(curRow, quotes);
206     end
207 end
208 end
209
210 function showDebug(A, stars, quotes, markedCols, markedRows)
211     for i = 1:size(A,1)
212         for j = 1:size(A,2)
213             if stars(i,j)
214                 fprintf("%d* \t", A(i,j));
215             elseif quotes(i,j)
216                 fprintf("%d' \t", A(i,j));
217             else
218                 fprintf("%d \t", A(i,j));
219             end
220         end
221         if i <= length(markedRows) && markedRows(i)
222             fprintf(" +\n");
223         else
224             fprintf("\n");
225         end
226     end
227     for j = 1:length(markedCols)
228         if markedCols(j)
229             fprintf("+ \t");
230         else
231             fprintf(" \t");
232         end
233     end
234     fprintf("\n");
235 end

```

2.2 Результаты решений

2.2.1 Задача минимизации

Листинг 2.3 – Вывод программы при задаче минимизации

```

1
2 Вычитание минимальных элементов по столбцам:
3     0     3     0     1     5
4     7     8     2     0     3

```

5		2	4	7	5	0
6		3	0	3	5	1
7		2	2	6	6	0
8						
9	Вычитание минимальных элементов по строкам:					
10		0	3	0	1	5
11		7	8	2	0	3
12		2	4	7	5	0
13		3	0	3	5	1
14		2	2	6	6	0
15						
16	Начальные 0*:					
17	0*	3	0	1	5	
18	7	8	2	0*	3	
19	2	4	7	5	0*	
20	3	0*	3	5	1	
21	2	2	6	6	0	
22						
23	k =					
24		4				
25						
26	k нужно улучшить.					
27	Итерация 1					
28	После нахождения 0'					
29	0*	3	0'	1	5	
30	7	8	2	0*	3	
31	2	4	7	5	0*	
32	3	0*	3	5	1	
33	2	2	6	6	0	
34	+	+		+	+	
35	Изменили выделенные строки и столбцы					
36	0*	3	0'	1	5	+
37	7	8	2	0*	3	
38	2	4	7	5	0*	
39	3	0*	3	5	1	
40	2	2	6	6	0	
41		+		+	+	
42	Пересчитали матрицу h					
43	0*	5	0'	3	7	+
44	5	8	0	0*	3	
45	0	4	5	5	0*	

```

46 1    0*   1    5    1
47 0    2    4    6    0
48      +      +    +
49 После нахождения 0'
50 0*   5    0'   3    7    +
51 5    8    0    0*   3
52 0'   4    5    5    0*
53 1    0*   1    5    1
54 0    2    4    6    0
55      +      +    +
56 Изменили выделенные строки и столбцы
57 0*   5    0'   3    7    +
58 5    8    0    0*   3
59 0'   4    5    5    0*   +
60 1    0*   1    5    1
61 0    2    4    6    0
62      +      +
63 После нахождения 0'
64 0*   5    0'   3    7    +
65 5    8    0    0*   3
66 0'   4    5    5    0*   +
67 1    0*   1    5    1
68 0'   2    4    6    0
69      +      +
70 Новые 0* после построения L-цепочки:
71      0      0      1      0      0
72      0      0      0      1      0
73      0      0      0      0      1
74      0      1      0      0      0
75      1      0      0      0      0
76
77 X_opt =
78      0      0      1      0      0
79      0      0      0      1      0
80      0      0      0      0      1
81      0      1      0      0      0
82      1      0      0      0      0
83
84 f_opt =
85      15

```

2.2.2 Задача максимизации

Листинг 2.4 – Вывод программы при задаче максимизации

```

1 Преобразование матрицы для задачи на максимум
2      7      5      8      6      2
3      0      0      6      7      4
4      5      4      1      2      7
5      4      8      5      2      6
6      5      6      2      1      7
7
8 Вычитание минимальных элементов по столбцам:
9      7      5      7      5      0
10     0      0      5      6      2
11     5      4      0      1      5
12     4      8      4      1      4
13     5      6      1      0      5
14
15 Вычитание минимальных элементов по строкам:
16     7      5      7      5      0
17     0      0      5      6      2
18     5      4      0      1      5
19     3      7      3      0      3
20     5      6      1      0      5
21
22 Начальные 0*:
23 7      5      7      5      0*
24 0* 0      5      6      2
25 5      4      0* 1      5
26 3      7      3      0* 3
27 5      6      1      0      5
28
29 k =
30     4
31
32 k нужно улучшить.
33 Итерация 1
34 После нахождения 0'
35 7      5      7      5      0*
36 0* 0' 5      6      2
37 5      4      0* 1      5
38 3      7      3      0* 3

```

39	5	6	1	0	5	
40	+		+	+	+	
41	Изменили выделенные строки и столбцы					
42	7	5	7	5	0*	
43	0*	0'	5	6	2	+
44	5	4	0*	1	5	
45	3	7	3	0*	3	
46	5	6	1	0	5	
47			+	+	+	
48	Пересчитали матрицу h					
49	4	2	7	5	0*	
50	0*	0'	8	9	5	+
51	2	1	0*	1	5	
52	0	4	3	0*	3	
53	2	3	1	0	5	
54			+	+	+	
55	После нахождения 0'					
56	4	2	7	5	0*	
57	0*	0'	8	9	5	+
58	2	1	0*	1	5	
59	0'	4	3	0*	3	
60	2	3	1	0	5	
61			+	+	+	
62	Изменили выделенные строки и столбцы					
63	4	2	7	5	0*	
64	0*	0'	8	9	5	+
65	2	1	0*	1	5	
66	0'	4	3	0*	3	+
67	2	3	1	0	5	
68			+		+	
69	После нахождения 0'					
70	4	2	7	5	0*	
71	0*	0'	8	9	5	+
72	2	1	0*	1	5	
73	0'	4	3	0*	3	+
74	2	3	1	0'	5	
75			+		+	
76	Новые 0* после построения L-цепочки:					
77		0		0		1
78		0		1		0
79		0		0		1

80	1	0	0	0	0
81	0	0	0	1	0
82					
83	X_opt =				
84	0	0	0	0	1
85	0	1	0	0	0
86	0	0	1	0	0
87	1	0	0	0	0
88	0	0	0	1	0
89					
90	f_opt =				
91	42				

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы был реализован венгерский метод решения задачи о назначениях для задач минимизации и максимизации. В ходе выполнения лабораторной работы была написана программа на языке MatLab, позволяющая применять венгерский метод решения задачи о назначениях